

这是一个很好的想法。结合我们今天花了几个小时踩过的坑（Cursor 3.9.16 + Windows + Python 3.13 + PowerShell），我建议这份指南不要照搬网上旧教程，而是以**实战**为主。

下面是我建议的目录。

《Cursor 3.9（2026最新版）Python 开发环境配置指南》

适用环境

软件	版本
Windows	11
Cursor	3.9.x
Python	3.13.x
PowerShell	7.x / Windows PowerShell
Git	最新版

第一章 安装Python

1.1 下载Python

推荐官网安装最新版Python。

安装时必须勾选

```
Add Python to PATH
```

安装完成验证：

```
python --version
```

例如：

```
Python 3.13.14
```

再执行

```
python -m pip --version
```

第二章 创建Cursor项目

推荐目录

```
Documents
├── projects
│   └── TA_workflow
```

Cursor:

```
File
  Open Folder
```

打开

```
TA_workflow
```

注意

不要打开

```
Documents
```

不要打开

```
projects
```

一定打开项目根目录。

第三章 创建Python虚拟环境

项目根目录执行

```
python -m venv .venv
```

生成

```
TA_workflow
├──
├── .venv
├──
├── Core
├──
└── requirements.txt
```

Python官方推荐使用 `venv` 创建隔离环境。([Python documentation](#))

第四章 激活虚拟环境

PowerShell

```
.\.venv\Scripts\Activate.ps1
```

成功后

```
(.venv)  
PS C:\Users\...
```

验证

```
python -c "import sys;print(sys.executable)"
```

应该输出

```
...\TA_Workflow\.venv\Scripts\python.exe
```

第五章 Cursor终端验证

不要使用

```
where python
```

建议使用

```
Get-Command python
```

输出应类似：

```
Application  
python.exe  
  
Source  
  
...\TA_Workflow\.venv\Scripts\python.exe
```

验证 pip:

```
python -m pip --version
```

验证包:

```
python -m pip list
```

第六章 安装项目依赖

推荐

```
python -m pip install -r requirements.txt
```

不要直接使用

```
pip install ...
```

统一使用

```
python -m pip
```

避免多Python环境导致安装到错误位置。

第七章 requirements.txt 最佳实践

不要立即使用

```
pipreqs
```

原因：

容易误识别

例如

```
HTMLParser  
thread  
xmlrpclib  
js
```

这些都不是Windows Python项目需要安装的依赖。

建议：

先安装真正需要的软件包

例如：

```
python -m pip install numpy pandas akshare yfinance matplotlib openpyxl requests
```

最后再生成：

```
python -m pip freeze > requirements.txt
```

这样得到的是当前环境的真实依赖。

第八章 运行Python程序

项目根目录

```
TA_workflow
```

运行

```
python .\Core\HK_Macro_History.py
```

不要进入

```
Core
```

再运行。

统一在项目根目录运行。

第九章 常见错误

1

```
ModuleNotFoundError
```

原因：

包未安装。

检查：

```
python -m pip show numpy
```

2

```
running scripts is disabled
```

PowerShell策略。

执行：

```
Set-ExecutionPolicy -Scope Process Bypass
```

然后重新：

```
.\.venv\Scripts\Activate.ps1
```

```
python 指向系统Python
```

检查:

```
python -c "import sys;print(sys.executable)"
```

如果不是

```
.venv
```

说明虚拟环境未激活。

第十章 Cursor 3.9 使用建议

根据我们今天的实践，针对 **Cursor 3.9.16**：

- 使用 **File** → **Open Folder** 打开项目根目录。
- 优先使用 **Cursor 内置 Terminal**，确保在同一终端中激活 `.venv`。
- 每次运行程序前，用 `python -c "import sys; print(sys.executable)"` 快速确认当前解释器。
- 如果 AI Agent 运行 Python 时出现缺少依赖，先确认它是否使用了项目的虚拟环境。社区中也有用户反馈 Agent 模式可能没有使用正确的虚拟环境。([Cursor - Community Forum](#))

附录 A：推荐的项目目录

```
TA_workflow/  
|  
├── .venv/  
|  
├── Core/  
|   ├── HK_Macro_History.py  
|   ├── HK_Index.py  
|   └── ...  
|  
├── Data/  
|  
├── Config/  
|  
├── Logs/  
|  
├── requirements.txt  
├── README.md  
└── .gitignore
```

附录 B：推荐的开发工具

对于你的 **TA_Workflow (港股量化 + AkShare + Python)**，我建议后续逐步加入：

- Git + GitHub (版本管理)
- Jupyter Notebook (数据分析)
- Ruff (代码检查与格式化)
- pytest (自动化测试)
- GitHub Copilot 或 Cursor AI (智能辅助开发)
- LangGraph / MCP (AI Agent workflow)

这样可以形成一个完整、可维护的 Python 量化开发环境。